



Checkpointing Strategies for Multi-Node Systems

Keeping LLM Training Safe,
Resumable, and Efficient

A technical guide for ML engineers and infrastructure architects

Why Checkpointing Matters

Multi-node jobs run for **weeks on 100s–1000s of GPUs**, making failures inevitable:

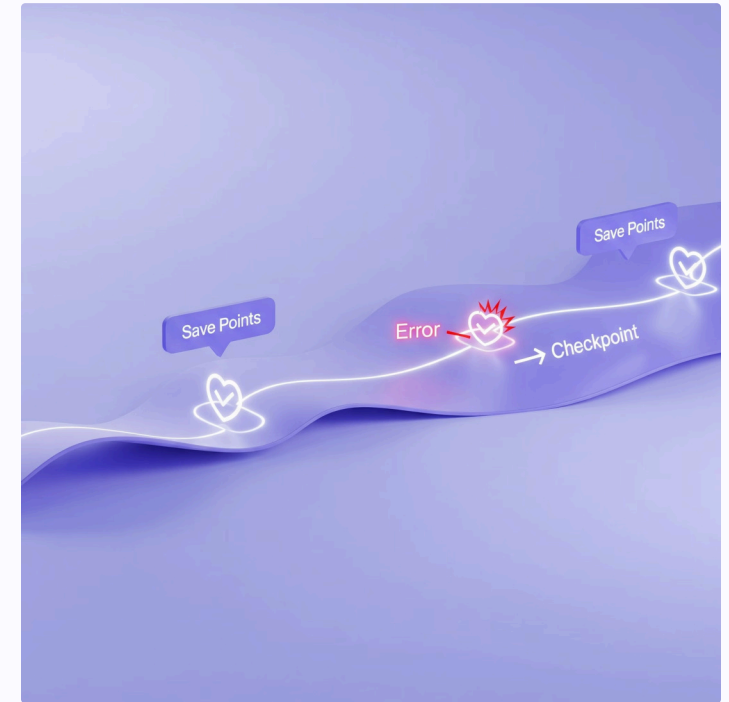
Node crashes

Network stalls

Preemptions

Hardware failures

Without proper checkpointing: **restart from scratch**



Goal: Fast, reliable recovery with minimal compute loss

What to Save

Model Parameters

Trained weights that define the model's behavior

Optimizer States

Adam moments typically require **2× the space of parameters** (first and second moments)

Gradients

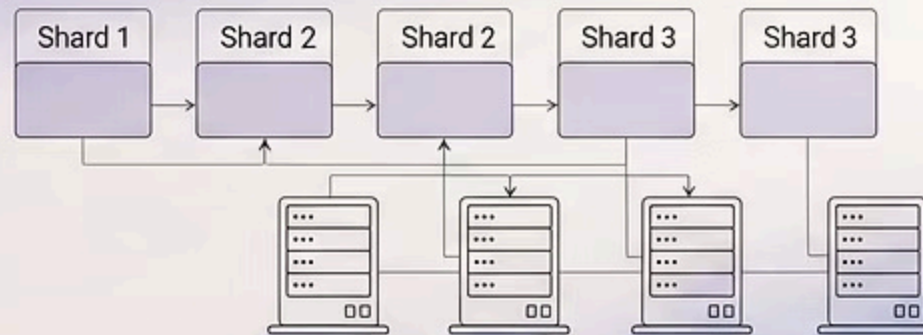
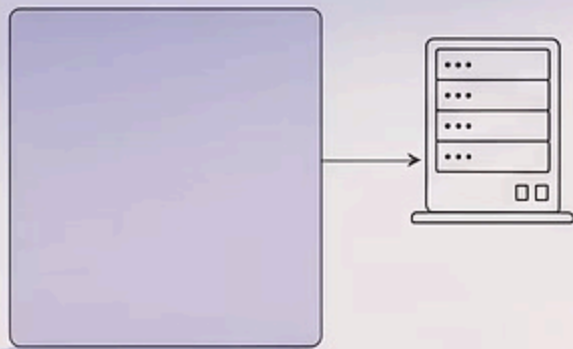
May be needed for specific training setups (pipeline parallelism, gradient accumulation)

RNG States

Critical for deterministic behavior and reproducibility across runs

Training Metadata

Epoch, step count, config, tokenizer, learning rate schedule state



Full vs Sharded Checkpoints

Full Checkpoint

- One giant file containing all model state
- Simple to implement and manage
- Becomes unwieldy for very large models
- Single point of failure
- Creates I/O bottlenecks at scale

Sharded Checkpoint

- Each GPU/node saves its portion of the model
- Faster parallel saves and smaller per-file size
- Standard in distributed training frameworks
- Enables faster save/load operations
- Reduces memory spikes during checkpoint ops

Async & Incremental Checkpoints



Traditional (Sync)

Training pauses completely during checkpoint operations



Asynchronous

Checkpoint in background threads while training continues

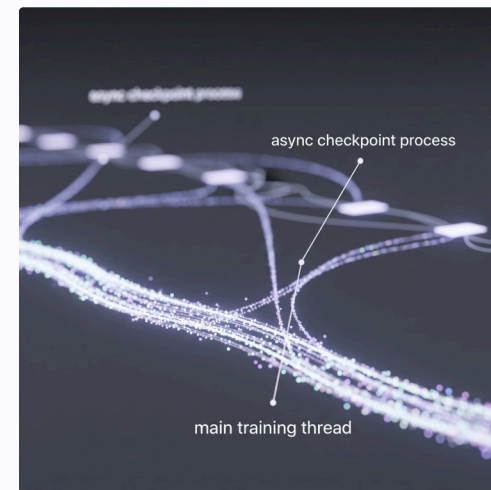


Incremental

Only save weights that changed since last checkpoint

Benefits:

- Reduces I/O bottlenecks, especially on large clusters
- Minimizes training interruptions
- Can achieve up to **95% reduction in checkpoint overhead**
- Commonly implemented in **DeepSpeed ZeRO-Infinity, PyTorch FSDP**



Storage Infrastructure

Distributed File Systems

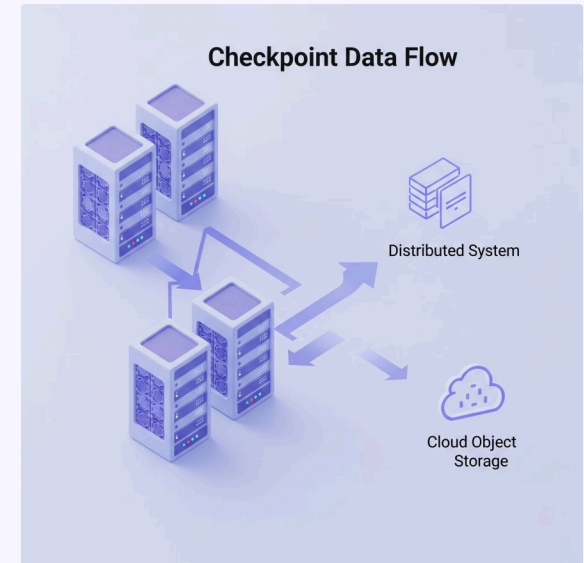
- Lustre, GPFS, BeeGFS
- High-throughput parallel access
- Optimized for HPC workloads

Cloud Object Storage

- S3, GCS, Azure Blob
- Infinite scale, high durability
- Multi-region replication

Key Requirements:

- High throughput (several GB/s) for parallel writes
- Redundancy with multi-region copies for safety
- Access controls and versioning for collaboration



Frequency Trade-Offs

1

Too Frequent

I/O bottleneck slows down training

Storage costs increase

Resource contention

2

Too Infrequent

High recompute cost after failure

Extended recovery time

Risk of missing data

Balanced Approach:

Rule of thumb: Save every **30-60 minutes** for long-running jobs

Use **lightweight evaluation checkpoints** more frequently + full saves less often

Adjust frequency based on failure rates in your environment

Fault Tolerance Enhancements



1 Checkpoint Retention

Keep $N-1$ previous checkpoints for rollback capability

2 Integrity Verification

Validate checkpoints (hashing) before overwriting "latest"

3 Automated Recovery

Implement retries on partial checkpoint writes

4 Orchestration Integration

Use job managers (Slurm/Kubernetes/TorchElastic) for automated recovery

Checkpointing in Practice

Real-world Examples from Large-scale LLM Training



i BLOOM's 350GB checkpoints would take ~1 hour to save on a 1GB/s connection, highlighting the importance of parallel I/O and asynchronous operations.

Best Practices

Use Sharded, Async Checkpointing

Especially for large jobs spanning multiple nodes

Consider memory vs. I/O trade-offs for your specific hardware

Implement Redundant Storage

High-throughput, multi-region with verification

Consider tiered storage (fast local + slower archive)

Automate Checkpoint-Restore Workflows

Script the entire process including validation

Test recovery procedures regularly

Validate Before Production

Test full recovery on small-scale before production runs

Measure checkpoint overhead as part of performance tuning



Regular testing of checkpoint recovery is critical for confidence in your fault tolerance system

Key Takeaways

Checkpoints = Insurance Policy

4+

Critical Components

Parameters, optimizer state, RNG state, and training metadata

2x

Efficiency Gain

Sharding + async saves provide at least double the checkpoint performance

30-60

Optimal Minutes

Recommended checkpoint frequency for balancing overhead vs. recovery

Robust checkpointing is the foundation of fault-tolerant infrastructure

Questions? Contact the ML Infrastructure Team